

[NOME DO AUTOR OU DA EQUIPE]

DOCUMENTAÇÃO TÉCNICA DO SISTEMA SPOT
GESTÃO INTEGRADA DE PROJETOS

[CIDADE]

2026

[NOME DO AUTOR OU DA EQUIPE]

DOCUMENTAÇÃO TÉCNICA DO SISTEMA SPOT
GESTÃO INTEGRADA DE PROJETOS

Documentação técnica apresentada à [NOME DA INSTITUIÇÃO OU ORGANIZAÇÃO], como registro da arquitetura, das funcionalidades, dos requisitos operacionais e da evolução do sistema Spot.

[CIDADE]

2026

RESUMO

Este documento registra o desenvolvimento do Spot, plataforma web destinada à organização, ao planejamento, à execução e ao acompanhamento financeiro de projetos. O sistema utiliza Laravel 13 no servidor, React 18 na interface, Vite no processo de construção, PostgreSQL para persistência e uma rotina Python com Pandas para consolidação analítica. A solução contém autenticação por sessão protegida por Laravel Sanctum, compatibilidade com autenticação JWT, login social, administração de usuários, segurança de conta, gestão de projetos e tarefas, apontamentos de horas, despesas, convites, agenda, caixa de entrada, integrações e relatórios financeiros. O levantamento foi elaborado a partir do código-fonte, das migrações, das rotas, dos testes automatizados e do histórico Git disponíveis em 4 de setembro de 2026. Foram identificados três marcos funcionais versionados e um conjunto local ainda não versionado, relativo principalmente ao painel de portfólios e ao processamento de indicadores com Python e Pandas. Na verificação executada para esta edição, os 52 testes automatizados foram aprovados, totalizando 262 asserções, e a construção da interface de produção foi concluída com sucesso. O documento adota organização compatível com normas ABNT aplicáveis à apresentação, numeração de seções, resumo, citações e referências, com campos institucionais mantidos para posterior preenchimento.

Palavras-chave: gestão de projetos; Laravel; React; segurança; relatórios financeiros; análise de portfólio.

ABSTRACT

This document records the development of Spot, a web platform for organizing, planning, executing, and financially monitoring projects. The system uses Laravel 13 on the server, React 18 on the user interface, Vite for asset building, PostgreSQL for persistence, and a Python/Pandas routine for analytical consolidation. The solution includes Sanctum-protected session authentication, JWT compatibility, social login, user administration, account security, project and task management, work logs, expenses, invitations, calendar, inbox, integrations, and financial reporting. The assessment was based on the codebase, migrations, routes, automated tests, and Git history available on September 4, 2026. Three versioned functional milestones and one unversioned local work set were identified. During verification, all 52 automated tests passed, totaling 262 assertions, and the production frontend build completed successfully.

Keywords: project management; Laravel; React; security; financial reports; portfolio analytics.

SUMÁRIO

1 INTRODUÇÃO

2 METODOLOGIA E DELIMITAÇÃO

3 VISÃO GERAL DO SISTEMA

4 ARQUITETURA E TECNOLOGIAS

5 FUNCIONALIDADES IMPLEMENTADAS

6 SEGURANÇA, IDENTIDADE E AUTORIZAÇÃO

7 MODELO DE DADOS

8 INTERFACES E CONTRATOS DE API

9 PROCESSAMENTO ANALÍTICO DE PORTFÓLIO

10 INSTALAÇÃO, CONFIGURAÇÃO E OPERAÇÃO

11 QUALIDADE E VERIFICAÇÃO

12 HISTÓRICO DE DESENVOLVIMENTO

13 LIMITAÇÕES E PENDÊNCIAS

14 CONCLUSÃO

REFERÊNCIAS

APÊNDICE A — INVENTÁRIO RESUMIDO DAS ROTAS

APÊNDICE B — RASTREABILIDADE DOS ARTEFATOS

1 INTRODUÇÃO

O Spot é uma aplicação de gestão integrada de projetos desenvolvida para centralizar informações de execução, responsáveis, prazos, tarefas, horas, despesas, comunicação operacional e resultados financeiros. Sua interface principal é uma aplicação de página única, enquanto o servidor expõe serviços HTTP responsáveis por autenticação, regras de negócio, persistência e autorização.

Esta documentação tem como objetivo consolidar o que foi produzido no repositório até a data de corte, tornar explícitas as decisões técnicas observáveis e oferecer uma referência para manutenção, auditoria, implantação e continuidade do desenvolvimento.

O texto não substitui especificações contratuais, políticas internas de segurança nem manuais de infraestrutura. Descreve o comportamento verificável no código disponível e separa funcionalidades versionadas de alterações locais ainda não incorporadas ao ramo principal.

2 METODOLOGIA E DELIMITAÇÃO

O levantamento foi realizado por inspeção do histórico Git, árvore de arquivos, configurações de dependências, rotas registradas pelo framework, migrações, modelos, controladores, ações de domínio, componentes de interface, testes e alterações locais. Foram também executados a suíte automatizada, o processo de construção do frontend e uma chamada direta ao processador analítico.

A data de corte é 4 de setembro de 2026, no ramo `main`, a partir do commit `8c0412a`, acrescido das modificações locais existentes no diretório de trabalho. A expressão “implementado” designa comportamento presente no código; “versionado” designa conteúdo registrado em commit; e “em andamento” designa conteúdo presente somente na árvore local.

Nota editorial: a composição segue, de forma adaptada a relatório técnico de software, os princípios de apresentação da ABNT NBR 14724, de numeração da ABNT NBR 6024, de sumário da ABNT NBR 6027, de resumo da ABNT NBR 6028, de citações da ABNT NBR 10520 e de referências da ABNT NBR 6023. Os campos de autoria, instituição e cidade permanecem marcados porque esses dados não constam de forma inequívoca no repositório.

3 VISÃO GERAL DO SISTEMA

3.1 Finalidade

A aplicação atende o ciclo operacional de projetos: cadastro, atribuição de equipe, definição de tarifas e tributos, acompanhamento de tarefas, registro de trabalho, controle de despesas, finalização ou reabertura governada e análise dos resultados. Recursos transversais incluem identidade, preferências, notificações, agenda, caixa de entrada e integrações com serviços Google e Microsoft.

3.2 Perfis e atores

O modelo de autorização utiliza perfis e permissões vinculados ao contexto empresarial. Os perfis administrativos observados são administrador e gestor-administrador. O segundo combina atribuições administrativas e operacionais. Gestores, analistas e convidados participam do fluxo conforme permissões e associação ao projeto. A aplicação impede a desativação ou o rebaixamento do último administrador ativo.

3.3 Escopo funcional consolidado

Quadro 1 — Macrofunções do Spot

Área	Capacidades observadas	Estado
Identidade	Login local, Google, Microsoft, primeiro acesso, recuperação de senha e perfil.	Versionado
Segurança	Sessões, CSRF, 2FA, provedores vinculados, preferências e permissões.	Versionado
Projetos	CRUD, membros, tarifas, impostos, ciclo de vida e exclusão lógica.	Versionado

Área	Capacidades observadas	Estado
Execução	Tarefas, apontamentos de horas, agenda, painel inicial e caixa de entrada.	Versionado
Finanças	Despesas, aprovação, resultado por projeto e consolidado de carteira.	Versionado
Estoque	Categorias, produtos, movimentações atômicas e painel legado.	Versionado/ compatibilidade
Portfólios	Indicadores, filtros, gráficos e detalhamento individual via Pandas.	Alteração local em andamento

Fonte: elaborado a partir do repositório Spot (2026).

4 ARQUITETURA E TECNOLOGIAS

4.1 Organização arquitetural

A solução combina uma aplicação Laravel e uma interface React no mesmo repositório. O Laravel entrega a página base Blade, mantém as rotas web e de API, acessa o banco de dados e executa regras de domínio. O React mantém navegação, formulários, painéis e interação do usuário. O Vite compila JavaScript e CSS para distribuição.

Regras sensíveis foram extraídas para classes de ação em `app/Domain`, entre elas auditoria, movimentação de estoque, revisão de despesas, cálculo financeiro, convites, apontamentos e transições do projeto. Essa divisão reduz a concentração de regras nos controladores e facilita testes.

4.2 Componentes tecnológicos

Quadro 2 — Componentes da solução

Componente	Versão/faixa	Responsabilidade
PHP	8.3 ou superior compatível	Execução da aplicação de servidor.
Laravel	13.8 ou compatível	API, persistência, validação, autorização e serviços web.
Laravel Sanctum	4.3 ou compatível	Autenticação da aplicação web por sessão e proteção CSRF.
JWT Auth	2.3 ou compatível	Compatibilidade da API anterior baseada em token.
Socialite	5.29 ou compatível	Autenticação social Google e Microsoft.

Componente	Versão/faixa	Responsabilidade
React	18.3.1	Interface de usuário em página única.
Vite	8.x	Servidor de desenvolvimento e construção dos ativos.
PostgreSQL	Banco relacional	Persistência padrão nos ambientes de desenvolvimento, testes e produção.
Python/Pandas	Python 3; Pandas ≥ 2.2 e < 3.0	Consolidação dos indicadores de portfólio.
PHPUnit	12.5 ou compatível	Testes automatizados.

Fonte: arquivos `composer.json`, `package.json` e `requirements.txt` (2026).

4.3 Fluxo de requisição

O navegador carrega a página Blade e os ativos compilados pelo Vite. A aplicação React consulta rotas sob `/api/v1`. Nas operações mutáveis, o cliente inicializa o cookie CSRF por `/sanctum/csrf-cookie` e envia a sessão de modo credenciado. O middleware autentica o usuário; o controlador valida os dados; autorizações e regras de domínio são aplicadas; e modelos Eloquent persistem ou consultam o banco. As respostas seguem o formato JSON.

5 FUNCIONALIDADES IMPLEMENTADAS

5.1 Autenticação e primeiro acesso

Não há cadastro público. O administrador provisiona a conta e gera um link de ativação de uso único. O usuário confirma os dados e define a senha. O reenvio do primeiro acesso é protegido para não revelar indevidamente a existência de contas. Google e Microsoft autenticam apenas usuários previamente autorizados, e cada conta pode manter, no máximo, uma identidade vinculada por provedor.

5.2 Administração e segurança de conta

A área administrativa permite criar usuários, alterar perfis e controlar o estado da conta. A área de segurança contempla alteração de senha, preferências, gerenciamento de sessões, desvinculação de provedores e configuração ou remoção de autenticação em dois fatores. Operações são condicionadas às regras de autorização.

5.3 Gestão de projetos

O CRUD da API principal exige número da proposta, status, situação e valor contratado. Clientes e gestores são validados no contexto da empresa Computécnica, conforme a regra presente. Dados financeiros são omitidos da resposta para usuários sem a permissão `financial.view`, permanecendo preservados no banco.

O ciclo de vida possui ações explícitas de finalização, reabertura e exclusão lógica. As ações exigem permissão, justificativa e registro de auditoria. Projetos finalizados rejeitam alterações e novas despesas, protegendo o estado consolidado.

5.4 Configuração, equipe, tarifas e tributos

A configuração reúne membros, funções, tarifas por analista e composição tributária. Tarifas possuem vigência e não podem se sobrepor. O lançamento de trabalho exige que o analista esteja atribuído ao projeto e possua tarifa vigente na data informada.

5.5 Horas, despesas e resultado financeiro

Ao registrar horas, o sistema armazena uma fotografia das tarifas de custo e venda, impedindo que reajustes futuros modifiquem resultados históricos. Horas extras podem adotar valores distintos para custo e faturamento. Despesas seguem fluxo de aprovação ou rejeição e somente itens aprovados compõem o resultado realizado.

O cálculo financeiro utiliza aritmética decimal. A receita realizada inclui horas extras faturáveis; os custos consideram as tarifas fotografadas; e a comissão pode incidir sobre o valor bruto ou sobre a receita líquida de impostos, conforme configuração.

5.6 Convites de projeto

Convites têm escopo restrito ao projeto, validade máxima de sete dias e uso único. O token completo é apresentado somente na criação; no banco é armazenado apenas o resumo SHA-256. Convites não utilizados podem ser revogados. Quando o e-mail já pertence a uma conta, o aceite exige autenticação nessa mesma conta.

5.7 Painel inicial, tarefas, agenda e caixa de entrada

O painel inicial reúne informações operacionais e tarefas. A aplicação permite criar, editar, alternar e remover tarefas. A agenda fornece listagem e criação de eventos. A caixa de entrada permite listar itens, marcar um item ou todos como lidos e arquivar registros. Preferências de notificação podem ser consultadas, substituídas ou restauradas.

5.8 Integrações e interface

A interface possui catálogo visual de aplicações Google e Microsoft, navegação lateral, menu superior, busca, filtros e perfil. Os arquivos de ícones foram organizados em `public/apps/integrations`. Alterações locais recentes ajustam a escala de ícones específicos, incluem favicon, centralizam a busca, acrescentam seletor de páginas e criam as telas de novo projeto e portfólios.

5.9 Estoque em camada de compatibilidade

A API de estoque oferece categorias, produtos, movimentos e dashboard. A quantidade de um produto não pode ser alterada diretamente pelo endpoint de atualização. O saldo muda somente por movimentação registrada na mesma transação do histórico; uma saída com saldo insuficiente recebe resposta HTTP 422. Essa camada utiliza JWT e permanece para compatibilidade.

6 SEGURANÇA, IDENTIDADE E AUTORIZAÇÃO

A SPA utiliza Laravel Sanctum, sessão e CSRF, evitando o armazenamento do token de autenticação em `localStorage`. A API principal aplica permissões por meio de gates. O isolamento empresarial é realizado com `current_company_id` nas consultas relevantes. Ações críticas geram trilha em `audit_logs`.

Os controles observados incluem validação de entrada, expiração e hash de convites, uso único de ativação, verificação de associação ao projeto, ocultação de campos financeiros, proteção do último administrador, autenticação em dois fatores e revogação de sessões. Para produção, são necessários HTTPS, cookies seguros, depuração desativada e armazenamento de sessão compartilhado quando houver mais de uma instância.

Permissões e testes reduzem riscos, mas não eliminam a necessidade de revisão periódica, gestão de segredos, limitação de requisições, monitoramento, cópias de segurança, atualização de dependências e teste de invasão no ambiente real.

7 MODELO DE DADOS

As migrações representam usuários, empresas, pessoas, clientes, perfis, permissões, projetos, tarefas, categorias, produtos, movimentos, membros, tarifas, tributos, despesas, apontamentos, convites, auditorias, tokens de ativação, sessões, agenda, caixa de entrada e integrações. O banco padrão de desenvolvimento é PostgreSQL, com um schema isolado para os testes automatizados.

Quadro 3 — Entidades centrais e relações

Entidade	Relações ou responsabilidade principal
User	Empresa atual, perfis, identidades sociais, segurança, projetos e preferências.
Project	Empresa, proprietário, gerente do projeto, tarefas, membros, horas, despesas, tributos e convites.
ProjectMember	Vínculo do usuário com projeto e função operacional.
AnalystProjectRate	Tarifas de custo e venda com vigência.
WorkLog	Horas, extras e fotografia dos valores aplicáveis.
ProjectExpense	Despesa, estado de revisão e participação no resultado.
ProjectTax/TaxType	Composição tributária do projeto.
ProjectInvitation	Convite temporário, escopo, resumo do token, aceite e revogação.
AuditLog	Registro de operações críticas e justificativas.
Product/	Saldo de estoque e histórico transacional.

Entidade	Relações ou responsabilidade principal
----------	--

StockMovement

Fonte: modelos e migrações do projeto Spot (2026).

8 INTERFACES E CONTRATOS DE API

8.1 API principal

A API principal utiliza o prefixo `/api/v1`. Rotas autenticadas cobrem sessão, projetos, tarefas, despesas, horas, convites, relatórios, parâmetros, usuários administrativos, segurança, notificações, agenda e caixa de entrada. O comando de inventário do Laravel registrou 105 rotas de aplicação na data de corte, incluindo rotas web e as duas famílias de API.

8.2 API legada

As rotas sem o prefixo `/v1` mantêm autenticação JWT e não devem ser escolhidas para novas integrações. Elas existem para evitar interrupção imediata de consumidores já implementados.

8.3 Convenções de resposta

A API retorna JSON e utiliza códigos HTTP compatíveis com o resultado da operação. Falhas de validação produzem respostas de entidade não processável; falhas de autenticação e autorização são tratadas pelos mecanismos do framework; indisponibilidade do processador analítico resulta em HTTP 503 com mensagem operacional e detalhe somente no ambiente local.

9 PROCESSAMENTO ANALÍTICO DE PORTFÓLIO

9.1 Estado da implementação

O painel analítico está presente como alteração local ainda não commitada. Ele inclui controlador invocável, rota protegida, relação com gerente do projeto, dependência Pandas, script de consolidação, teste funcional, componentes React, estilos, navegação e documentação de instalação no README.

9.2 Fluxo de processamento

O endpoint `GET /api/v1/reports/project-analytics` exige `financial.view` e aceita os filtros opcionais `from`, `to` e `project_id`. A consulta é limitada à empresa atual. O Laravel carrega os projetos e calcula o resultado financeiro com a mesma classe já utilizada pelos relatórios. Em seguida, envia um documento JSON pela entrada padrão a `scripts/project_analytics.py`.

O processo Python não acessa o banco. Com Pandas, normaliza números e datas, classifica projetos, agrega status, clientes e gerentes, cria série mensal, soma contrato, receita, custo e lucro e devolve o levantamento individual. O processo possui limite de 30 segundos e o executável pode ser configurado por `PYTHON_BINARY`.

9.3 Indicadores

O resumo apresenta total, em andamento, concluídos, no prazo, atrasados, congelados, quantidade de clientes e gerentes, valor contratado, receita, custo e lucro. A interface gera gráfico de situação, evolução mensal, quantidade por cliente, quantidade por gerente, consolidado financeiro e painéis individuais com progresso, horas e resultado.

10 INSTALAÇÃO, CONFIGURAÇÃO E OPERAÇÃO

10.1 Execução local simplificada

O repositório inclui runtimes locais e o script `iniciar.sh`. Em Linux, a inicialização é feita por:

```
./iniciar.sh
```

O FrankenPHP atende a porta 8000 e o Vite a porta 5173. A aplicação é acessada em `http://localhost:8000`. A interrupção dos processos ocorre com `Ctrl+C`.

10.2 Preparação

```
./composer install ./php artisan migrate --seed export PATH="$PWD/.runtime/bin:$PATH"
npm install npm run build python3 -m pip install --user -r requirements.txt
```

O uso de `--user` é adequado ao ambiente local descrito. Em produção, recomenda-se ambiente virtual isolado e definição explícita de `PYTHON_BINARY`.

10.3 Variáveis essenciais de produção

```
APP_ENV=production APP_DEBUG=false APP_URL=https://spot.seudominio.com
SESSION_SECURE_COOKIE=true SESSION_SAME_SITE=lax
```

Credenciais de banco, e-mail e provedores sociais devem ser configuradas fora do controle de versão. O arquivo `.env` não deve ser publicado.

11 QUALIDADE E VERIFICAÇÃO

11.1 Testes automatizados

A suíte cobre autenticação social, sessão, logout, recuperação de senha, perfil, avatar, segurança de conta, provisionamento administrativo, permissões, projetos, governança, configurações, convites, apontamentos, despesas, finanças, estoque, agenda, caixa de entrada, notificações e dashboard.

Quadro 4 — Resultado da verificação em 4 de setembro de 2026

Verificação		Resultado	Evidência
PHPUnit		Aprovado	52 testes; 52 aprovados; 262 asserções.
Build Vite		Aprovado	1.564 módulos transformados; CSS e JavaScript gerados.
Processador vazio	Pandas	Aprovado	Retorno JSON válido com totais zerados; Pandas 2.3.3.
Inventário de rotas		Aprovado	105 rotas registradas.

Fonte: execução local dos comandos do projeto (2026).

11.2 Limites da verificação

Os resultados confirmam o comportamento coberto pelos testes e a capacidade de compilar o frontend no ambiente disponível. Não constituem homologação completa de usabilidade, desempenho, acessibilidade, segurança ofensiva, provedores OAuth reais, entrega de e-mail ou implantação distribuída.

12 HISTÓRICO DE DESENVOLVIMENTO

Quadro 5 — Marcos registrados no Git

Data	Identificador	Conteúdo
9 jun. 2026	16bb9fa	Estrutura inicial Laravel, autenticação, interface, configurações, banco e protótipo desktop.
9 jun. 2026	1aaa0cf	Ajustes para retirar dependências geradas do versionamento.
9 jun. 2026	225a2a3	Remoção do projeto duplicado <code>spot-desktop</code> do repositório.
9 jun. 2026	2c1b8aa	Exclusão lógica de arquivos automaticamente gerados por regras de ignore.
20 ago. 2026	7400ff7	Arquitetura API JWT, estoque, projetos, tarefas, autenticação social, redefinição de senha, perfil e nova SPA.
23 ago. 2026	8c0412a	Governança de projetos, segurança por sessão, administração, finanças, horas, despesas, convites, parâmetros, agenda, caixa de entrada, notificações e integrações.
4 set. 2026	Árvore local	Painel de portfólios com Pandas, cadastro de projeto, menu de páginas, ajustes visuais e favicon; conteúdo ainda não versionado.

Fonte: histórico Git e estado do diretório de trabalho do Spot (2026).

13 LIMITAÇÕES E PENDÊNCIAS

Na data de corte, devem ser considerados os seguintes pontos:

- as alterações de analytics, interface, ícones e favicon ainda não estavam registradas em commit;
- a execução do painel de portfólios depende da disponibilidade do Python e do Pandas no servidor;
- a API JWT antiga aumenta a superfície de manutenção enquanto permanecer ativa;
- os campos de autoria, instituição e cidade deste documento requerem preenchimento;
- o sumário do PDF é editorial e deve ser atualizado caso haja paginação ou conteúdo acrescentado;
- homologações de acessibilidade, carga, integração OAuth e e-mail não foram comprovadas nesta rodada;
- o PostgreSQL deve ser operado com bancos isolados por ambiente, backups, monitoramento e alta disponibilidade compatíveis com a produção.

14 CONCLUSÃO

O repositório registra a evolução de um protótipo Laravel para uma plataforma integrada com API versionada, SPA React, controles de segurança, governança de projetos e apuração financeira. O uso de ações de domínio, auditoria, permissões e testes evidencia preocupação com consistência e rastreabilidade.

As verificações desta edição foram concluídas sem falhas. O principal trabalho ainda pendente de versionamento é o módulo analítico de portfólios e seus ajustes associados. Antes de uma liberação, recomenda-se revisar e agrupar essas alterações em commit próprio, instalar a dependência Python no ambiente-alvo e executar homologação funcional e de segurança.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação**. Rio de Janeiro: ABNT, 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023: informação e documentação — referências — elaboração**. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6024: informação e documentação — numeração progressiva das seções de um documento — apresentação**. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6027: informação e documentação — sumário — apresentação**. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6028: informação e documentação — resumo, resenha e recensão — apresentação**. Rio de Janeiro: ABNT, 2021.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520: informação e documentação — citações em documentos — apresentação**. Rio de Janeiro: ABNT, 2023.

LARAVEL. **Laravel documentation**. [S. l.], 2026. Disponível em: <<https://laravel.com/docs>>. Acesso em: 4 set. 2026.

META PLATFORMS. **React documentation**. [S. l.], 2026. Disponível em: <<https://react.dev/>>. Acesso em: 4 set. 2026.

OPENJS FOUNDATION. **Vite guide**. [S. l.], 2026. Disponível em: <<https://vite.dev/guide/>>. Acesso em: 4 set. 2026.

THE PANDAS DEVELOPMENT TEAM. **Pandas documentation**. [S. l.], 2026.

Disponível em: <<https://pandas.pydata.org/docs/>>. Acesso em: 4 set. 2026.

SPOT. **Repositório local do sistema Spot**: código-fonte, histórico Git, testes e arquivos de configuração. Versão consultada: commit 8c0412a e alterações locais. [S. l.], 2026.

APÊNDICE A — INVENTÁRIO RESUMIDO DAS ROTAS

Quadro 6 — Grupos de endpoints da API principal

Grupo	Operações principais
Autenticação	POST /api/v1/auth/login; POST /api/v1/auth/logout; GET /api/v1/auth/me; projetos do usuário; fusos horários.
Projetos	Listar, criar, exibir, alterar, excluir logicamente, finalizar, reabrir e consultar configuração.
Equipe e finanças	e Membros, tarifas, tributos, despesas, revisão, apontamentos e resultado financeiro.
Convites	Criar, aceitar e revogar.
Relatórios	GET /api/v1/reports/portfolio e GET /api/v1/reports/project-analytics.
Administração	Provisionar usuário e alterar perfil/estado.
Segurança	Senha, preferências, provedores, sessões e autenticação em dois fatores.
Operação	Dashboard, tarefas, agenda, caixa de entrada, notificações e parâmetros.
Estoque v1	Categorias, produtos, movimentos e dashboard sob /api/v1.

Fonte: routes/api.php e inventário do Laravel (2026).

APÊNDICE B — RASTREABILIDADE DOS ARTEFATOS

Quadro 7 — Localização dos principais artefatos

Finalidade	Localização
Regras de domínio	app/Domain/
Controladores HTTP	app/Http/Controllers/
Validação de projetos e estoque	app/Http/Requests/
Modelos de persistência	app/Models/
Esquema e evolução do banco	database/migrations/
Dados iniciais e demonstração	database/seeder/
Rotas	routes/api.php e routes/web.php
Aplicação React	resources/js/App.jsx
Estilos	resources/css/app.css
Página base	resources/views/app.blade.php
Analytics	app/Http/Controllers/Api/ProjectAnalyticsController.php e scripts/project_analytics.py
Testes	tests/Feature/ e tests/Unit/

Finalidade	Localização
Operação local	README.md, <code>iniciar.sh</code> , <code>composer.json</code> , <code>package.json</code> e <code>requirements.txt</code>

Fonte: repositório Spot (2026).